



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks

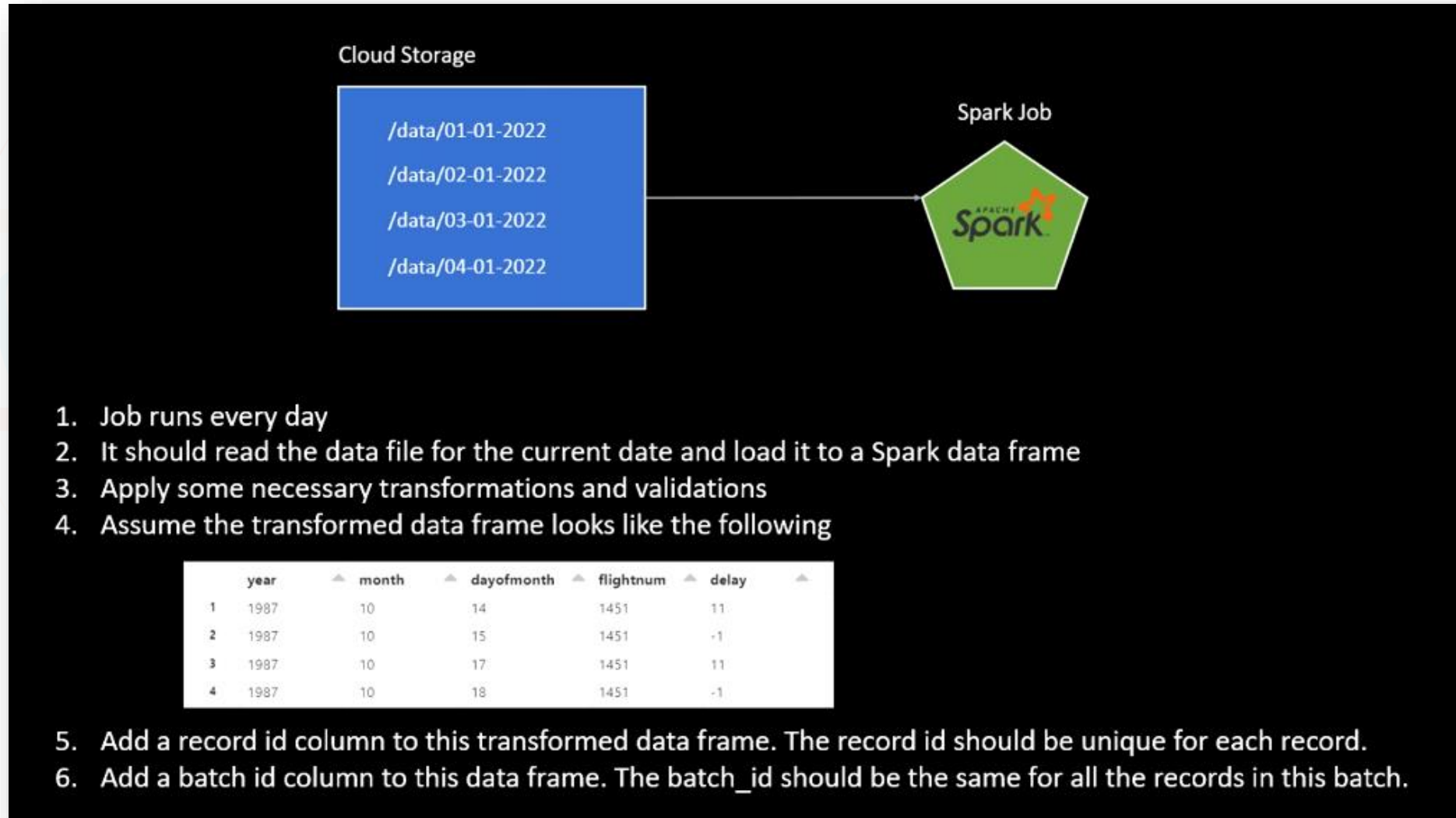


Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Working with Literal Values

We have a simple requirement. You receive data in a cloud storage directory. The data comes every day in a date-wise directory. You are asked to create a Spark batch job to do the following.



The first requirement is to run the job every day.

And running a Job every day requires scheduling your Spark notebook or your Application file to run daily at a scheduled time.

We are not going into the details of scheduling for now.

I will cover Job scheduling in the latter part of the course.

So leave that requirement for now.

The second requirement is to read the data file for the current date.

There are many ways to achieve this requirement.

One easy method is to have date-wise directories.

Parameterize the load() method to read data from the current-date directory.

I will also leave this requirement and show you an example in the coming lecture.

The third requirement is to apply some transformations.

Let's leave this one also for now. We are learning transformations throughout the course.

And the focus of this lecture is on the fifth and the sixth requirement.

So let's leave this requirement also.

The fourth one shows you a sample dataframe.

So we will start from this requirement.

I will create this sample dataframe for you.

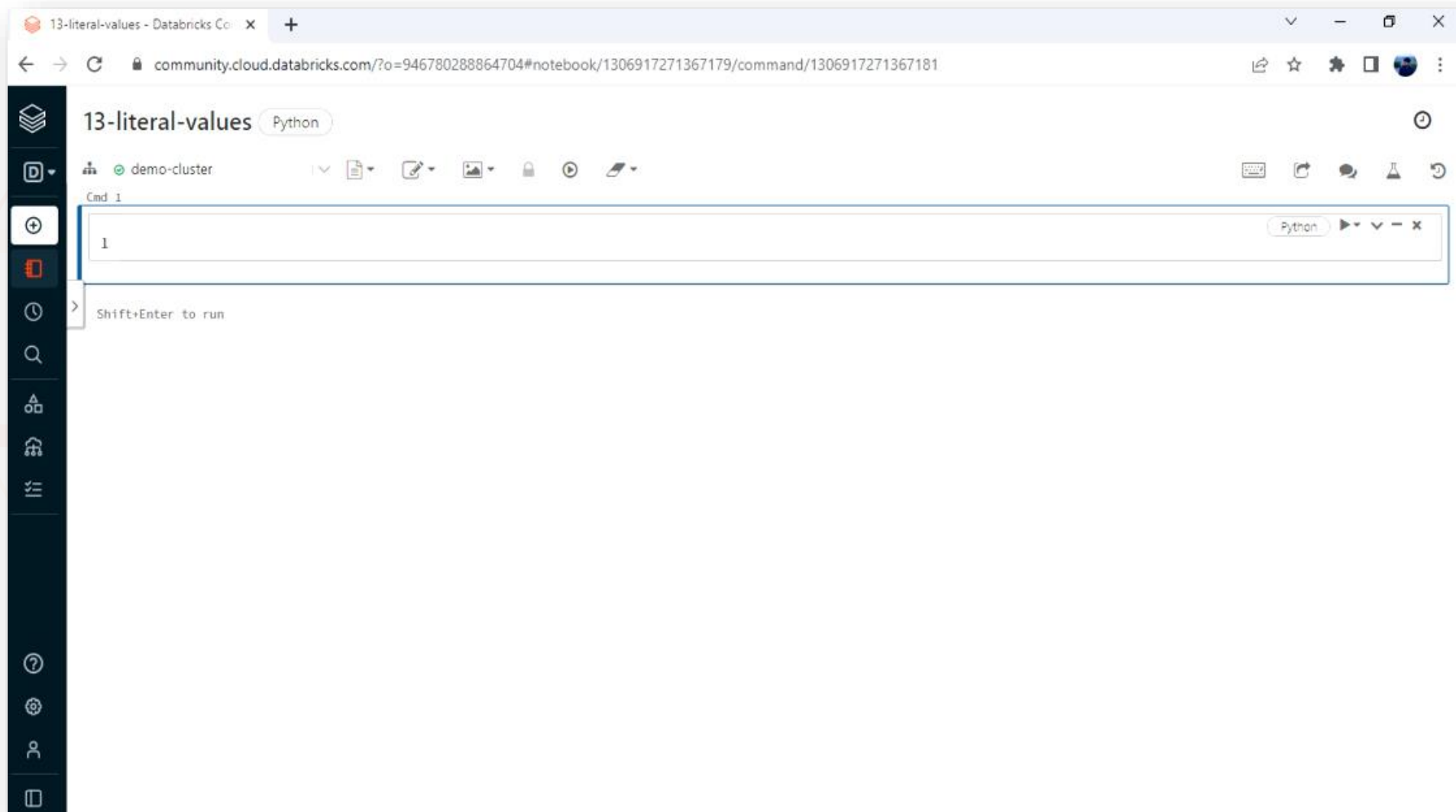
And you can learn how to implement the following requirements. Make sense?

Great! So our primary focus is on the fifth and sixth items.

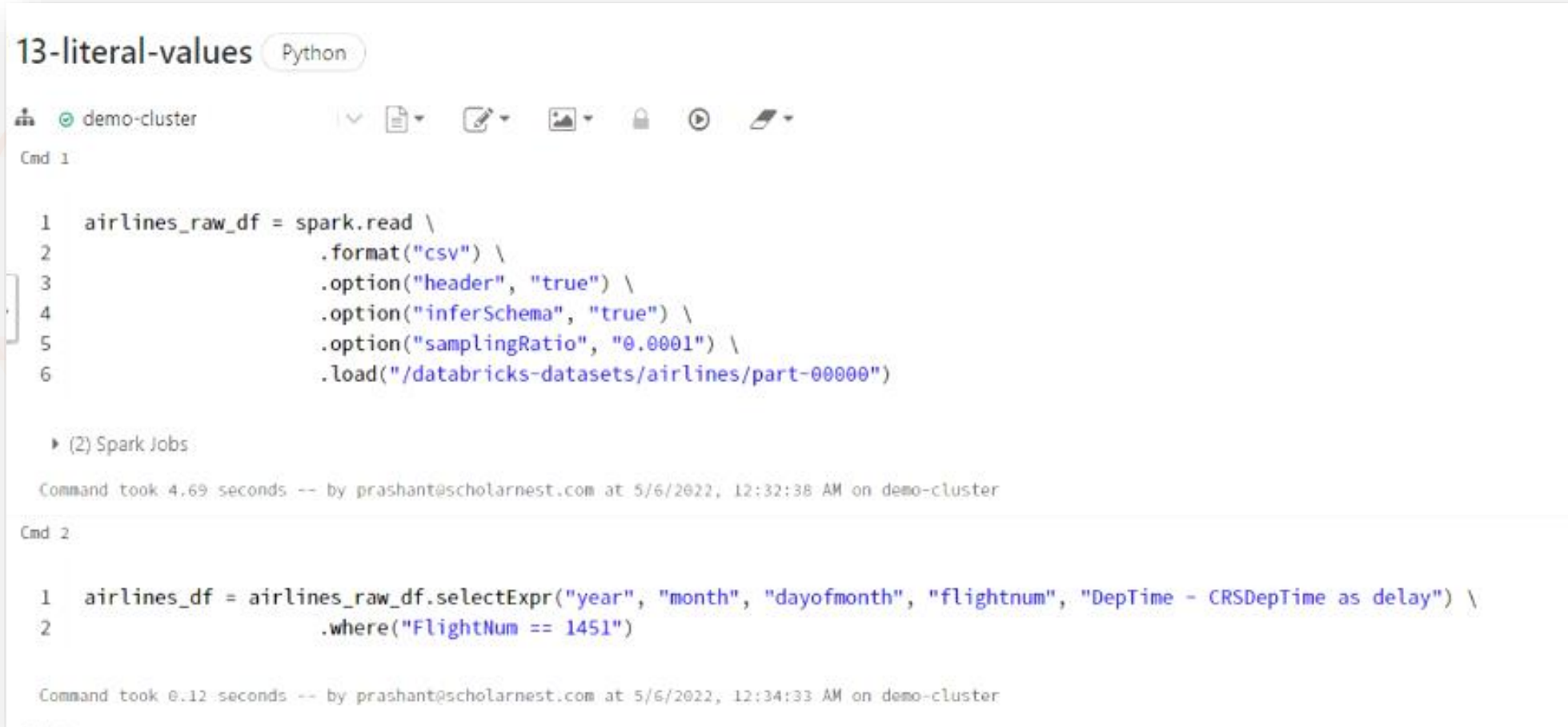
The fifth item wants us to add a unique record id for each record in the given dataframe.

Let's achieve this requirement, and then we will look into the last one.

Go to your Databricks workspace and create a new notebook. **(Reference : 13-literal-values)**



The first one is the same Dataframe that we created in our earlier lecture. We need to apply some transformations and prepare a Dataframe as per the example. So we transformed it and created a new Dataframe.



13-literal-values Python

demo-cluster

Cmd 1

```
1 airlines_raw_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .option("samplingRatio", "0.0001") \  
6     .load("/databricks-datasets/airlines/part-00000")
```

► (2) Spark Jobs

Command took 4.69 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:32:38 AM on demo-cluster

Cmd 2

```
1 airlines_df = airlines_raw_df.selectExpr("year", "month", "dayofmonth", "flightnum", "DepTime - CRSDepTime as delay") \  
2     .where("FlightNum == 1451")
```

Command took 0.12 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:34:33 AM on demo-cluster

We got the desired Dataframe. Now we have the following requirement:
Add a record id column to this transformed dataframe. The record id should be unique for each record.

Cmd 3

```
1 display(airlines_df)
```

▶ (3) Spark Jobs

	year	month	dayofmonth	flightnum	delay
1	1987	10	14	1451	11
2	1987	10	15	1451	-1
3	1987	10	17	1451	11
4	1987	10	18	1451	-1
5	1987	10	19	1451	19
6	1987	10	21	1451	-2
7	1987	10	22	1451	-2

Showing all 235 rows.

Command took 6.76 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:34:50 AM on demo-cluster

How to do it? Can we use some function that generates a unique id for us?

We have two places to look for the available functions.

1. Built-in Spark SQL functions
2. Dataframe functions.

I checked both the places and identified two potential functions that we can use.

1. `uuid()`
2. `monotonically_increasing_id()`

The `uuid()` function is a built-in spark SQL function.

It generates a 32-character unique id.

We can use it to generate a UUID for each record.

The `monotonically_increasing_id()` is a dataframe function.

It generates a long number starting from 0.

We can use this function also to add a unique id for our records.

The `monotonically_increasing_id()` is a dataframe function. So you must import it. Once imported, you can use the `withColumn()` transformation to add a new column to your dataframe. The `withColumn()` takes two arguments. The first argument is the column name, and the second argument is the logic. The logic is as simple as using a function and generating a `monotonically_increasing_id`. You can see in the output there is the `id` column at the end, it starts with zero and goes on increasing by one.

13-literal-values

Python

demo-cluster

Command took 6.76 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:34:50 AM on demo-cluster

Cmd 4

```
1 from pyspark.sql.functions import monotonicly_increasing_id
2
3 airlines_df.withColumn("id", monotonicly_increasing_id()) \
4     .show(5, False)
```

(1) Spark Jobs

year	month	dayofmonth	flightnum	delay	id
1987	10	14	1451	11	0
1987	10	15	1451	-1	1
1987	10	17	1451	11	2
1987	10	18	1451	-1	3
1987	10	19	1451	19	4

only showing top 5 rows

Command took 0.54 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:40:32 AM on demo-cluster

We have another alternative of using the UUID() built-in function. So I added another column and used the UUID() built-in function. Since the UUID() is a built-in function, I must use the expr() function to evaluate it. And we can see a unique id for each record. You can use any one of these depending upon your requirement.

13-literal-values

Python

demo-cluster

Command took 6.76 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:34:50 AM on demo-cluster

Cmd 4

```
1 from pyspark.sql.functions import monotonically_increasing_id, expr
2
3 airlines_df.withColumn("id", monotonically_increasing_id()) \
4               .withColumn("RecordID", expr("uuid()")) \
5               .show(5, False)
```

(1) Spark Jobs

year	month	dayofmonth	flightnum	delay	id	RecordID
1987	10	14	1451	11	0	bbfb4a69-2c6e-4f6e-a695-27edf83ccc74
1987	10	15	1451	-1	1	506c57dc-c092-4c5e-b555-5acb7905919f
1987	10	17	1451	11	2	b84dcefd-777c-40e2-94fa-c29b3e7bd769
1987	10	18	1451	-1	3	12fed1e8-7b7e-42a7-a699-5cde16d11e8e
1987	10	19	1451	19	4	5cf3a766-ac8e-40f7-9479-8dbba2e7f03f

only showing top 5 rows

Command took 0.52 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:48:35 AM on demo-cluster

We want to add a batch id column to this dataframe. The batch_id should be the same for all the records in this batch. So it is like a hardcoded id for all the records.

How will you do it?

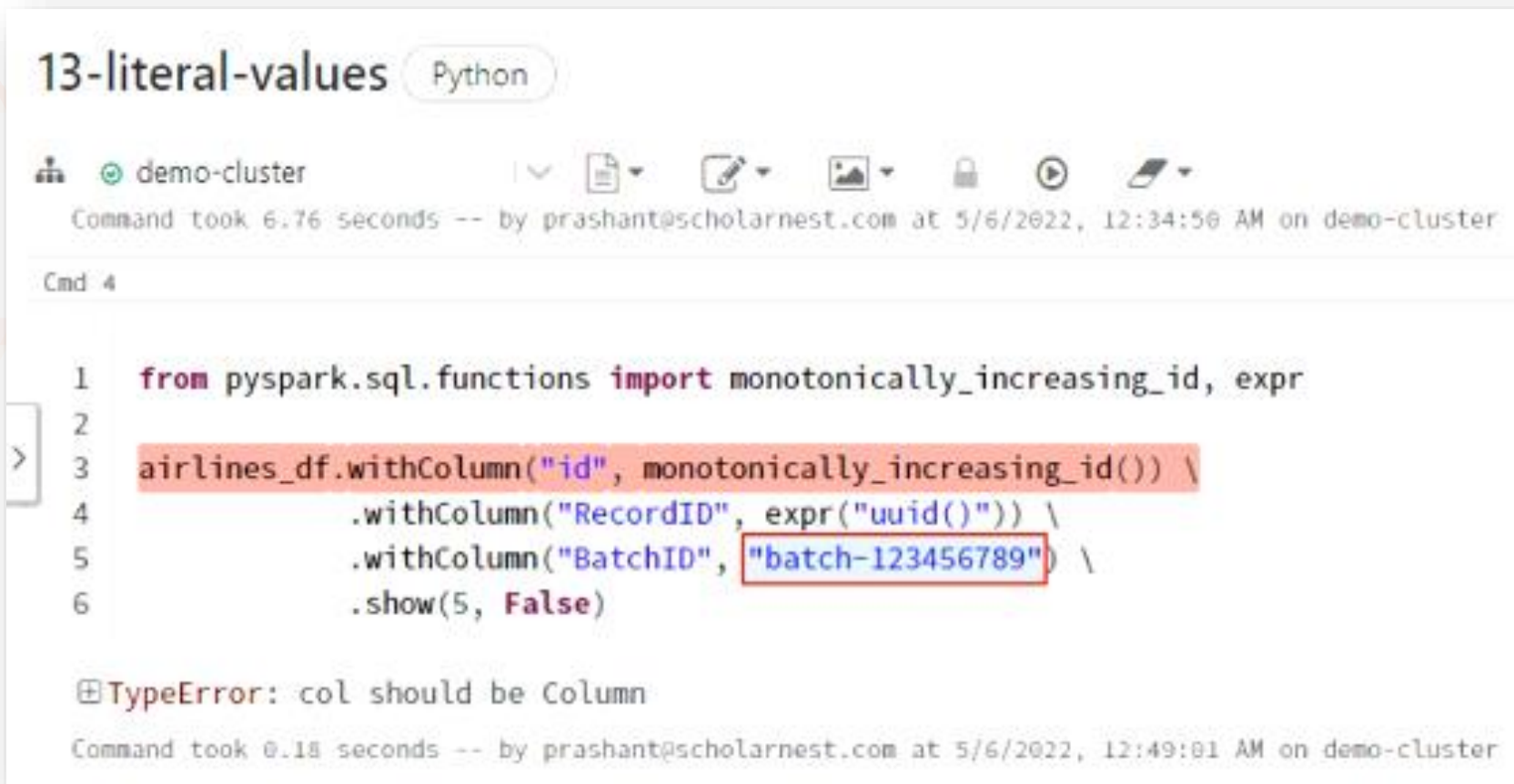
Can you even think of a SQL way of doing it?

Assume you have a table. You want to select all columns and add one extra column with a hardcoded value.

Here is a SQL expression to do it:

```
SELECT *, "batch-123456789" as batchID from table_name;
```

I wanted to add a new column to my Dataframe, so I used them with column transformation. The column name is BatchID, and the logic to create a column is a literal value. But if we try running it, we will get an error. Because the Dataframe doesn't understand the literal value. The second argument to the withColumn() must be an expression that evaluates a column object. This string is not an expression. I can wrap it with an expr() function, but even the expr() function will not evaluate it. Because the string here is a literal value. Expressions are created using column names, functions, and operators. And this string doesn't represent any column name. It is neither a function. It is a literal value, and Dataframe API cannot understand literal values until we explicitly tell it.



The screenshot shows a Jupyter Notebook interface with a title bar '13-literal-values' and a 'Python' language selector. Below the title bar, it indicates the command was executed on a 'demo-cluster' and took 6.76 seconds. The command is labeled 'Cmd 4'. The code snippet is as follows:

```
1 from pyspark.sql.functions import monotonically_increasing_id, expr
2
3 airlines_df.withColumn("id", monotonically_increasing_id()) \
4             .withColumn("RecordID", expr("uuid()")) \
5             .withColumn("BatchID", "batch-123456789") \
6             .show(5, False)
```

The code is executed, and a `TypeError: col should be Column` error is displayed. The error message is preceded by a small icon of a box with a plus sign. Below the error message, it indicates the command took 0.18 seconds.

I can import a Python UUID package and create a unique id to use the same example. Then I can use the python uid variable inside the lit() function.
So instead of hardcoding a manually created id, I can use a variable to automatically generate a unique id and use it as my batch id.

13-literal-values Python

demo-cluster

Command took 6.76 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:34:50 AM on demo-cluster

Cmd 4

```
1 from pyspark.sql.functions import monotonically_increasing_id, expr, lit
2
3 import uuid
4 uid = str(uuid.uuid4())
5
6 airlines_df.withColumn("id", monotonically_increasing_id()) \
7     .withColumn("RecordID", expr("uuid()")) \
8     .withColumn("BatchID", lit(uid)) \
9     .show(5, False)
```

(1) Spark Jobs

year	month	dayofmonth	flightnum	delay	id	RecordID	BatchID
1987	10	14	1451	11	0	5836314d-3702-4aa7-bd69-06ed0265ba97	932aed37-4837-4eaf-883c-540cc06c1132
1987	10	15	1451	-1	1	aec77f55-2cfc-4c7b-bc65-bcf2a45f4bfd	932aed37-4837-4eaf-883c-540cc06c1132
1987	10	17	1451	11	2	6dee1ebf-a0dd-4f43-b3d9-b2ed6b0260c9	932aed37-4837-4eaf-883c-540cc06c1132
1987	10	18	1451	-1	3	0fc12fc1-d6e5-4280-9923-b88ee368e008	932aed37-4837-4eaf-883c-540cc06c1132
1987	10	19	1451	19	4	8591b94f-ad50-4edb-ba25-91709408b2cc	932aed37-4837-4eaf-883c-540cc06c1132

only showing top 5 rows

Command took 0.67 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:51:01 AM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com